DressMe

Speaker script

8 LangGraph agents, including one conversational orchestrator

4 FastAPI services, 1 cited RAG, A2A 0.3.0 protocol

Control plane / data plane, inputs-outputs, jury questions

Technical presentation - 18 minutes - 15 slides

Final-year project - React 19, FastAPI, LangGraph, OpenAI, Supabase

How to use this script

Total time: 18 minutes — 14 minutes of talk and 4 minutes of demo.

Slides: 15. One idea per slide. The slide is the evidence; you are the argument. Never read a slide aloud.

Cues: SHOW = what is on screen. **SAY** = the spoken text to adapt, not to recite. **MOVE** = the transition sentence that carries you to the next slide.

Golden rule: every technical claim must end on a benefit or a trade-off. A jury remembers "why", not "what".

Backup: the presentation PDFs (`DressMe_Presentation_EN.pdf`) contain the figures and the full detail if a question goes deep.

Model accuracy note — say this if asked. The eight agents do not all run on the same model. BO1 agents and the Trip Agent run on **GPT-4o** by default (`LLM_MODEL=gpt-4o`). The standalone 8070 orchestrator and BO4 use the local **qwen2.5:3b** through Ollama. The Purchase decision agent runs a **fine-tuned Qwen2.5-3B + LoRA**, GPT-4o only rewrites the verdict, and XGBoost is the deterministic fallback. Present this as a deliberate choice: the right model for the right job, not one model everywhere.

Part 1 — The talk (0:00–14:00)

0:00 — Opening (30 s)

SHOW: title slide with the one-line subtitle.

SAY: "DressMe turns a phone photo into a structured wardrobe, then eight LangGraph agents reason on real garments, real weather and cited fashion books to answer five questions: what do I wear, what outfits can I build, what do I pack, what am I missing, and should I buy this? The rule is simple: the model decides and writes; deterministic code verifies and computes."

MOVE: "Let me start with why this is not just another wardrobe gallery."

0:30 — Problem and promise (45 s)

SHOW: three pain points: invisible wardrobe, no combination logic, irrational packing and purchase.

SAY: "Existing apps are galleries. They show clothes but they do not reason about them. DressMe adds three things: a structured wardrobe, grounded reasoning, and deterministic solvers. And we make three promises: never invent a garment, never invent weather, never invent a source."

MOVE: "Here is what that looks like on screen."

1:15 — Product walkthrough (60 s)

SHOW: three screenshots: AI Stylist answer with weather, outfit and `[S#]` sources; Outfit Builder with three outfits; Help Me Pack checklist.

SAY: "Every block you see is backed by real data. The stylist answer carries the weather and the sources it used. The outfit cards only use garments the user owns. The packing checklist is validated by a solver, not written freely by the model. The backend also returns `agents_called`, which gives us a trace of which agent answered."

MOVE: "Under the interface, four independently deployable services."

2:15 — Global architecture (75 s)

SHOW: Figure 1 — services and ports.

SAY: "React 19 talks to BO1 on 8000 for wardrobe, stylist, similar and outfit, to BO3 on 8001 for gap analysis, to Help Me Pack on 8020 for trips, and to the RAG service on 8010 for fashion knowledge. Supabase provides

authentication, Postgres and storage. OpenAI and Open-Meteo are the only external providers. Two local alternatives exist: the standalone orchestrator on 8070 and the BO4 stylist on 8061; they are not wired to React."

MOVE: "The architecture only matters if the data is safe."

3:30 — Data, auth and memory (45 s)

SHOW: Figure 2.

SAY: "Every request carries a Supabase JWT. A FastAPI dependency resolves the user, and every SQL query is filtered by that user id. The same token is forwarded on A2A tasks, so an agent cannot read another user's wardrobe. Conversation memory is a LangGraph checkpointer; wardrobe corrections expire after fifteen minutes if the user does not confirm."

MOVE: "The first step of the pipeline is vision."

4:15 — Vision pipeline (45 s)

SHOW: Figure 3.

SAY: "YOLO runs first for geometry and colour, measured on pixels. GPT-4o vision runs second for semantics, with strict JSON and a fixed taxonomy of twenty classes. If the model is unavailable, a deterministic fallback still produces a valid garment. The reason is simple: the model must not guess the colour, and it must not invent a garment that is not in the photo."

MOVE: "Once the wardrobe exists, eight agents work on it."

5:00 — The eight agents (60 s)

SHOW: agent table: agent, role, model.

SAY: "One orchestrator and seven specialists. The AI Stylist answers the open question; Wardrobe owns the data and the writes; Similar decides owned, close or missing; Outfit Builder composes; Trip plans and packs; Gap analyses; Purchase decides. Each agent has a narrow contract, its own tools and its own evaluation. GPT-4o for the BO1 agents and Trip, the GPT-4o family for Gap, fine-tuned Qwen plus LoRA for the purchase decision, local qwen for the non-wired alternatives."

MOVE: "But an agent is not a model call. It is a graph."

6:00 — LangGraph: why (75 s)

SHOW: Figure 4, then the Wardrobe `gate` → `apply / cancel / assist` graph.

SAY: "LangGraph is not decoration. It gives us five things a plain function does not: explicit shared state, conditional edges, a checkpointer for per-user conversation memory, interrupts for `input-required`, and a shared tool-loop pattern reused by every agent. Look at Wardrobe: the gate decides whether to apply, cancel or assist; that is a state machine, not an if-statement. Look at Outfit Builder: validation loops back to composition. Look at Trip: two graphs, one conversational and one deterministic."

SAY: "What LangGraph is not for: YOLO, CP-SAT, XGBoost and RAG retrieval. Those are deterministic engines and they stay plain code. The rule is: graph where there is stateful control flow, plain code where there is an exact algorithm."

MOVE: "Now the part that surprises people: the orchestrator does not own every message."

7:15 — Orchestrator: the limited role (75 s)

SHOW: Figure 14 — control plane vs data plane.

SAY: "The orchestrator is the control plane. It routes each user turn with GPT-4o JSON plus French and English rules, it keeps the conversation state, it delegates to stylist, wardrobe, outfit, trip or gap over A2A, and it fetches

the wardrobe snapshot to pass to the agents. That is user intent."

SAY: "It is not a universal message bus. Capability calls happen on the data plane. The AI Stylist and the Similar Agent call the Wardrobe Agent in-process for a read-only lookup. The Trip Agent calls the Gap Agent directly over A2A through `check_gaps`. Non-chat routes use `call_with_wardrobe` to hand the wardrobe to an agent without entering the chat graph. If we routed all of this through the chat orchestrator, it would become a god object, a bottleneck and a single point of failure."

MOVE: "So how do agents talk when they are not in the chat flow? With A2A."

8:30 — A2A: why (90 s)

SHOW: Figure 13 — A2A cards and task lifecycle.

SAY: "A2A 0.3.0, JSON-RPC 2.0. Every agent publishes a public card at `/.well-known/agent-card.json`. The methods are `message/send`, `tasks/get` and `tasks/cancel`. Task states are submitted, working, input-required, completed, failed, canceled and rejected. The token travels with the task, and a task is readable only by its creator."

SAY: "Why not ad-hoc REST? Because A2A gives us discovery, a standard lifecycle, standard errors and interoperability. The same contract is used by the orchestrator, by a script, and by another agent. Concrete example: Trip sends its capsule to Gap; if Gap is down, the trip continues with a clean error instead of crashing. That is the standard doing real work."

MOVE: "Let me make the contracts concrete."

10:00 — Inputs and outputs (40 s)

SHOW: I/O cheat sheet on the slide.

SAY: "Every feature has one explicit JSON request and one explicit response. The response carries real item ids, `sources` when knowledge is used, and `decision_source` when a verdict is produced. That is what keeps the UI decoupled and the tests honest."

MOVE: "The knowledge layer is what makes the advice defensible."

10:40 — RAG and algorithms (75 s)

SHOW: Figure 15 and the algorithm table.

SAY: "Thirty-eight PDFs, 14,574 chunks, all-MiniLM-L6-v2 embeddings, hybrid search with reciprocal rank fusion, top five passages. Answers cite `[S#]` and refuse when the context is insufficient. The measured numbers: Hit@5 91.67 percent, MRR 85.42 percent, context precision 71.67 percent."

SAY: "Around the RAG, deterministic algorithms do the exact work: YOLO for detection, OR-Tools CP-SAT for the suitcase, XGBoost as a purchase baseline, a transparent outfit score — colour 0.30, style 0.20, occasion 0.30, season 0.20 — FashionCLIP and CLIP for visual similarity. The model where judgment is needed; the algorithm where correctness is needed."

MOVE: "A system is only credible if it fails well."

11:55 — Quality, guardrails, tests (45 s)

SHOW: guardrail table.

SAY: "Grounding on real ids, real weather and cited sources. Code validation on top of the model: outfit rules, Pydantic plus the solver for trips, checked purchase verdicts. Graceful degradation everywhere: if an agent is down, the caller gets a clean error or a fallback. And honesty: the model says 'I don't know' instead of inventing, and we disclose that the XGBoost baseline is trained on synthetic data."

SAY: "Tests: 101 on the orchestrator, 85 on the backend, 30 on the frontend, 1,095 on BO4 with 25 skipped, plus end-to-end trials."

MOVE: "No system is finished. Here is what is not."

12:40 — Limits and roadmap (45 s)

SHOW: limits table.

SAY: "RAG does not know the wardrobe; the two are deliberately separate. XGBoost is synthetic until we collect real feedback. Trip memory is in-process. BO4 and 8070 are not wired to React. A2A has no streaming or push yet. Roadmap: per-user preference memory, a worn-outfit journal, A2A streaming, and a fully A2A orchestrator."

MOVE: "Let me close."

13:25 — Conclusion (30 s)

SHOW: the closing sentence on screen.

SAY: "DressMe is a grounded multi-agent wardrobe: eight LangGraph agents, four services, a cited RAG, A2A and deterministic solvers. The orchestrator decides who answers; LangGraph decides how each agent answers; A2A lets agents ask each other; and code guarantees the answer is true."

MOVE: "I will show you the five flows in the product, then take your questions."

Part 2 — The four justifications (answers to memorize)

Why LangGraph?

Because the difficult parts are stateful control flow: confirmation gates, validation loops, two-graph trip planning, conversation memory and resuming after `input-required`. LangGraph makes those explicit and observable. It is deliberately not used for YOLO, CP-SAT, XGBoost or retrieval.

Why A2A?

Because agents are services, not functions. A2A gives discovery, a standard task lifecycle, authentication and a contract reusable by the orchestrator, a script or another agent. Ad-hoc REST would work, but you would rebuild cards, states and errors.

Why does the orchestrator not route everything?

Because it is the control plane, not the data plane. Routing every internal read through it creates a bottleneck, a god object and tight coupling. Capability calls are direct and read-only; writes stay with the owning agent.

Why hybrid instead of one central brain?

Because the system needs both global intent and local autonomy. The orchestrator decides who answers; each agent decides how to answer with its own tools and guardrails. That is what makes agents independently testable and deployable.

Part 3 — The five communication flows

- **User chat.** React calls `POST /api/assistant/chat`; the orchestrator routes, calls one agent over A2A, and the `respond` node writes the answer into the checkpointed thread.

- **Wardrobe capability.** The AI Stylist and the Similar Agent call `wardrobe_agent.ask()` in-process; the call is read-only.
- **Trip to Gap.** `check_gaps` sends the trip capsule to the Gap Agent over A2A with the user token.
- **Trip to Wardrobe.** The Trip Agent lists wardrobe items over A2A, with a BO1 route as fallback.
- **Non-chat routes.** `call_with_wardrobe()` performs the Wardrobe to agent handoff directly, outside the chat orchestrator.

Say this sentence: "The orchestrator coordinates user intent; agents carry their own capabilities."

Part 4 — Inputs and outputs (cheat sheet)

- **AI Stylist** — in: `message`, `city`, `country`, `unit`; out: `answer`, `weather`, `outfit`, `why`, `alternatives`, `tips`, `sources`, `items`.
- **Wardrobe Agent** — in: `message`, optional `action` and `confirm`; out: `reply`, `items`, `filters`, `proposal`.
- **Similar Agent** — in: `pieces` over A2A; out: per piece `status` (owned, close, missing), `match_item`, `missing_spec`.
- **Outfit Builder** — in: `occasion`, `season`, `city`, `avoid`; out: `outfits` with `title`, `items`, `weather_note`, `why`, plus `missing` and `sources`.
- **Trip Agent** — in: `message`, `session_id`; out: `message`, `state`, `capsule`, `checklist`, `alternatives`.
- **Gap Agent** — in: `message`, `goal`; out: `answer`, `gaps`, `covered`, `products`.
- **Purchase Agent** — in: `image`; out: `decision`, `score`, `reasons`, `matching_item_ids`, `decision_source`.
- **Avatar pipeline** — in: `photo` or `outfit_id`; out: `job_id`, `status`, `result_url`.

Part 5 — Jury questions

"Why not one big agent?"

Separation of concerns, data ownership, independent evaluation and a smaller blast radius. Wardrobe owns the wardrobe; Purchase owns the verdict; Trip owns the suitcase.

"Why LangGraph instead of plain loops?"

State, conditional edges, checkpointing, interrupts and a reusable tool loop. You would otherwise rewrite all five.

"Why A2A instead of REST?"

Discovery, task lifecycle, interoperability and one contract for every client. REST would work, but you would rebuild the card, the states and the errors.

"How do you prevent hallucination?"

Real item ids, real weather, cited sources, code validation, deterministic fallbacks, and an explicit "I don't know".

"Why a fine-tuned Qwen for purchase and GPT-4o elsewhere?"

The purchase decision is narrow and repetitive: a small LoRA model is cheaper and controllable. GPT-4o writes the explanation. XGBoost is the deterministic fallback. The main agents need broad reasoning, so they use GPT-4o.

"Is the orchestrator a bottleneck?"

For chat it is the entry point by design; capability calls bypass it. Each service scales horizontally, and the expensive resource is the LLM call, not the routing node.

"Why keep BO4 and 8070 if they are not wired?"

They are honest local, no-key variants kept for comparison. We present them as not wired rather than pretending they are part of the product.

"What was the hardest part?"

Trip `input-required` continuity and the CP-SAT constraints; then the Wardrobe write confirmation, because a wrong write is worse than a wrong answer.

"How do you evaluate the agents?"

Unit tests with fake LLM, RAG and weather; integration tests; end-to-end trials; RAG metrics; and a trace of `agents_called`.

"What would you do with three more months?"

Preference memory, a worn-outfit journal, real purchase feedback for XGBoost, A2A streaming and a fully A2A orchestrator.

Part 6 — Demo script (4 minutes)

1. Add three garments by photo; show the generated record and the measured colour.
2. Wardrobe: "show me my jackets for work"; show the LLM-chosen filters.
3. AI Stylist: "what should I wear tomorrow evening for a dinner in Tunis?"; show weather, outfit, reasons and sources.
4. Outfit Builder: build three outfits, then shuffle.
5. Help Me Pack: "Paris, June 3 to 7, 8 kg"; show the clarification, then the checklist.
6. Gap Agent: show ranked gaps and Tunisian products.
7. Purchase: show the verdict, the evidence and the fallback.
8. Finish with `python scripts/a2a_discover.py --ask` to show the cards and a real task.

If a service is down, say what the fallback is instead of hiding it. That is part of the story.

Part 7 — Closing, checklist and short version

Timing summary

- 0:00 — Opening — 30 s
- 0:30 — Problem and product — 1 min 45
- 2:15 — Architecture, data and vision — 2 min 45
- 5:00 — Agents, LangGraph and orchestrator — 3 min 30
- 8:30 — A2A, inputs-outputs and RAG — 3 min 25
- 11:55 — Quality, limits and conclusion — 2 min 05
- 14:00 — Demo — 4 min
- 18:00 — End

Closing line

"The orchestrator decides who answers; LangGraph decides how each agent answers; A2A lets agents ask each other; and deterministic code guarantees the answer is true."

Pre-flight checklist

- Five services started: BO1 8000, BO3 8001, RAG 8010, Packing 8020, frontend 8443.
- Supabase session valid and at least ten garments in the demo account.
- One stylist question, one outfit, one trip already tested before the jury.
- `DressMe_Presentation_EN.pdf` open as backup.
- `A2A_TOKEN` exported if you plan to run `a2a_discover.py --ask`.

If you only have 8 minutes

Keep the opening, the architecture, the orchestrator control-plane slide, the A2A slide and the conclusion. Drop the vision, the I/O slide, the quality slide and the demo; mention that the PDF contains them. State the four justifications in one sentence each.

Final sentence before questions

"Everything you saw is either a real database row, a real API response, or a deterministic computation. Ask me about any layer."